Sign Alphabet

A Gesture Recognition System for ASL Alphabet Learning

Abstract

This report introduces *Sign Alphabet*, a gesture recognition system designed for learning the American Sign Language (ASL) alphabet. Sign language, a visual modality with its own grammar and syntax, is a crucial means of communication for the deaf and hard-of-hearing community. This project explores the intersection of sign language learning and multimodal interaction by developing a digital system that employs gestures, clicks, and visual output for learning sign language communication, with a focus on the ASL alphabet. The sign language recognition system underwent several steps, starting with creating a dataset of images for each ASL letter. The dataset was processed using MediaPipe Hands. The Random Forest machine learning algorithm was employed for classification, achieving a 99.6% accuracy on the test dataset. The model was integrated into the program, providing real-time gesture recognition. The GUI was implemented using Tkinter, and OpenCV for the camera display. To evaluate the program, a usability test was conducted with four participants. The think-aloud method was employed, and participants answered interview questions to provide insights into their experience. This led to a reiteration of the design. In conclusion, the program does fulfill the goal of being easy and effective to use for learning the letters of ASL. However, there are some possible improvements and ideas for future research which are discussed.

Introduction

Sign language is a visual modality that conveys linguistic information through hand gestures, body language, and facial expressions. It is a distinct and fully developed language, with its own grammar and syntax, and was developed for deaf and hard-of-hearing individuals to communicate, as well as for those who want to communicate with them. There are many different sign languages around the world, and each has its own unique set of signs and rules. American Sign Language (ASL) is one of the most well-known sign languages, primarily used in the United States and Canada. Just like spoken languages, sign languages aren't universal; they evolve in separate communities and thus have regional variations (Reagan, 2007). More than 70 million people around the world use sign language to communicate, although it is hard to quantify the number of people who speak sign language as the proficiency levels can vary massively. They are not only useful for people with hearing deficiencies but also for people with autism and Down syndrome (Foggetti, 2022).

In recent years, it has become increasingly popular to learn sign language for people who are not in direct need of it to communicate. At the same time, using gestures for communication has become a zone of interest in the trailblazing field of multimodal interaction, which aims to redefine the way people interact with digital systems. This project aims to explore the

narrow intersection between these two topics through the development of a digital system called Sign Alphabet. It was developed to encourage sign language learning for people regardless of their need to learn it and lower the threshold for starting to learn. This system is a digital, multimodal fingerspelling application that uses clicking and gestures as input modalities and visual cues from a user interface as the output modality.

With Sign Alphabet, our central goals were to develop a multimodal system that teaches the user basic sign language intuitively and enjoyably, and secondly to showcase the strengths of using gestures as a modality for communicating with digital systems.

1. Background

1.1 Sign Language Pedagogy

In numerous countries, the absence of standardized sign language pedagogy leaves teaching methods to the discretion of instructors. Sign language education varies based on whether it is the student's first language (L1), or second language (L2). Regardless of language status, the curriculum typically progresses from basic everyday vocabulary and grammatical structures to more complex elements, encompassing culture, history, beliefs, and art. Notably, in the L1 curriculum, both teachers and students employ sign language to teach and learn academic subjects, establishing a foundation for deaf children to grasp other subjects. Fingerspelling, a manual representation of written words, is encouraged as the first step in learning sign language since it serves as a tool for new signers lacking vocabulary (Rosen, 2019).

Prior studies have demonstrated the effectiveness of games in facilitating fingerspelling learning. In a study by Joy et al. (2019) they introduced SignQuiz, an affordable online application designed for learning finger-spelling in ISL (Indian Sign Language). This application utilizes automatic sign language detection, offering a resource-free method for mastering signs. The outcomes reveal that SignQuiz surpasses traditional printed materials in the learning process.

Using a digital application with automatic sign language recognition to learn fingerspelling has other benefits as well. Firstly, you don’t need a tutor to get feedback on your fingerspelling. Secondly, learning through games is considered fun by many. By lowering thresholds to start learning and making it fun to learn sign language, we hope to make using sign language known and accepted among different audiences.

1.2 Existing Systems for Sign Language Recognition

Recognizing sign language is a complex computer vision task. The process usually involves five stages as shown in Figure 1: image acquisition, image preprocessing, segmentation, feature extraction, and classification (Adeyanju et al., 2021). The following paragraphs will walk through the most commonly used techniques and tools in each of the five stages.

Figure 1: The five stages of recognizing sign language

In the image acquisition step, the most commonly used method to capture images of signs is a camera or a webcam, as they are easy to use and low cost. Other methods include using a glove with sensors that extract the features needed for sign recognition, the Microsoft Kinect, or a Leap Motion Controller (S.M. Kamal et al., 2019).

The next step according to Adeyanju et al. (2019) is usually preprocessing the images to enhance quality and remove noise. The preprocessing techniques can be divided into two categories. The first one is called image enhancement, which aims to increase the image quality for machine analysis. Examples include histogram equalization, adaptive histogram equalization, and logarithmic transformation. The second one is image restoration, used to reduce unwanted noise, achieved by filtering the image through different filters such as mean filters, median filters, or Gaussian filters.

The next step is image segmentation, where regions of the image with meaningful information are detected. This can be done in several ways. In research, the most common methods include thresholding, edge-based segmentation, region-based segmentation, clustering, and applying artificial neural networks (Adeyanju et al., 2019).

Feature extraction is then applied to find the most relevant features of an image. The most commonly used method is Principal Component Analysis (PCA), but there are also others such as Fourier descriptors and Scale Invariant Feature Transformation (Adeyanju et al., 2019).

Finally, the processed and extracted features need to be classified for the corresponding sign. Different machine learning methods are used to achieve this, such as k-nearest neighbors, artificial neural networks, convolutional neural networks, and support vector machines (SVMs) (Adeyanju et al., 2021).

In one of the more recent studies a system recognizing signs from the Indian sign language was developed, translating signs to text. A dataset with 2194 images of eleven different signs was generated and preprocessed using Google's Mediapipe Holistic model, tracking landmarks of the face, body pose, and hands, for feature extraction. Three different classification models were then trained and tested: Decision Tree Classifier, Random Forest Classifier, and Gradient Boosting Classifier. When evaluated, five different parameters were considered: training accuracy, testing accuracy, precision, recall, and F1 score, as well as a comparison of confusion matrices derived for each model. As a result, it was found that the Random Forest model performed with the highest score on all parameters. With an accuracy of 97.4%, the authors claim that this proposed model, using Mediapipe for feature extraction and Random Forest for classification, is the highest-performing model when compared with other recently developed systems for ISL recognition (Adhikary et al., 2021).

2. Method

2.1 Development of the Program

2.1.1 Graphical User Interface

The process of creating the program's GUI started with brainstorming ideas and storyboarding as seen in Figure 2.

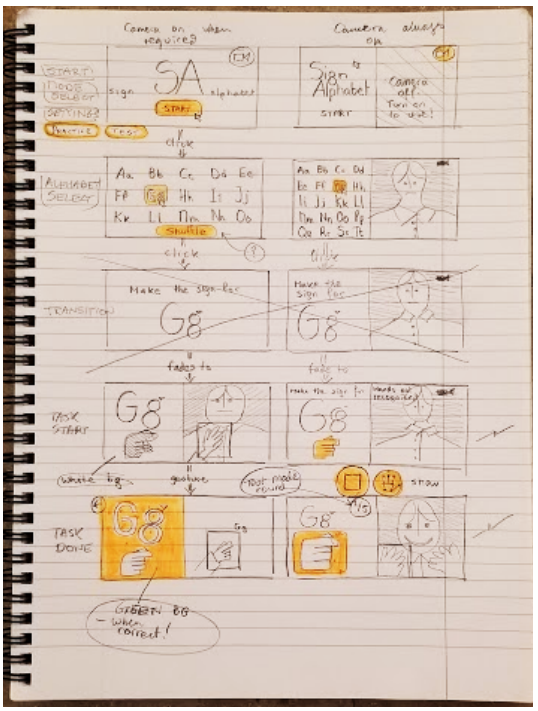

Figure 2: First storyboard

A Figma prototype was created with inspiration taken from instructional posters as seen in the mood board in Figure 3. This decision was made to create a simple and informative design that allows the user to focus on the task of learning sign language without distractions or disruptions.

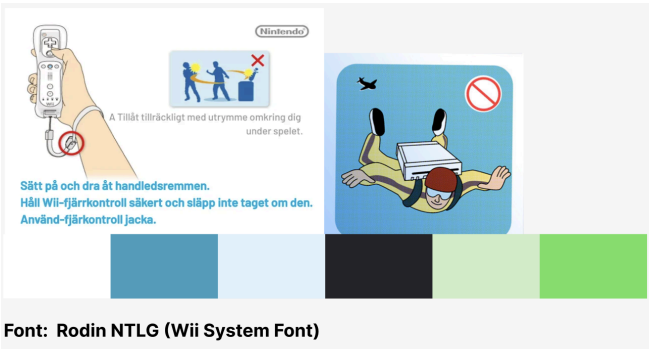

Figure 3: Moodboard in Figma

After the color palette, font, and "feel" of the program were decided, a simple prototype was created as seen in Figures 4-7. The screen is split into halves on all pages of the program, where one half is always displaying the user through the computer's webcam. On the *Home page*, shown in Figure 4, the user has the option of picking between two modes: they can either practice sign language (*Practice mode*) or test their abilities (*Test mode*). Figures 5 and 6 display the Practice mode, where the user can pick a letter of the alphabet to practice (5, *Alphabet page*), and be taken to a page that shows the sign for the specific letter (6, *Practice page*). A square is displayed around their hand if their hand is on the screen to give feedback to the user that the display has registered their palm. This square turns green and displays "Correct!" and the letter that was detected when the user's gesture made the correct sign. This is done to give visual feedback to the user. Figure 7 shows a prototype for the *Test page*, where the user gets assigned 10 random letters and has to sign them in under 2 minutes. This decision was made to give the program the feeling of a game, hopefully motivating the users in their learning. After the prototype was complete, the program was developed using Tkinter (Python Software Foundation, 2019) for the GUI and OpenCV for the camera display.

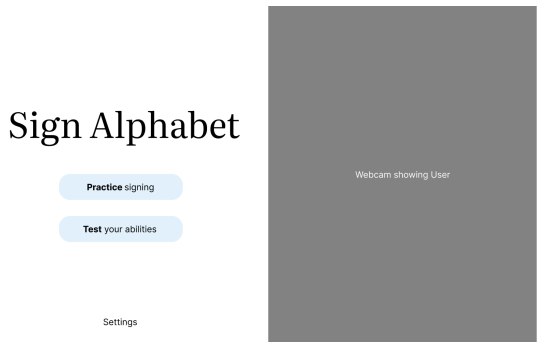

Figure 4: Prototype of the Home page

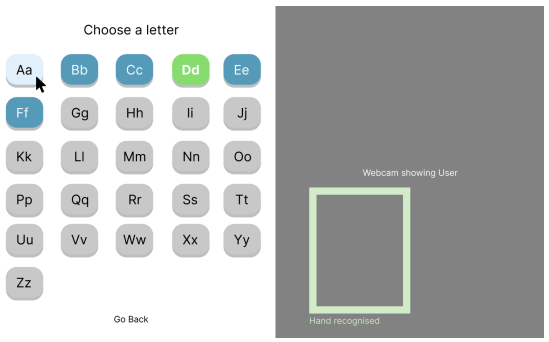

Figure 5: Prototype of the Alphabet page

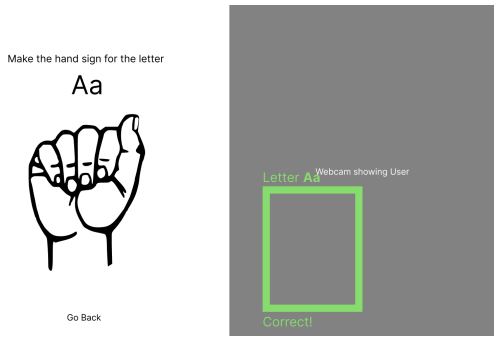

Figure 6: Prototype of the Practice page

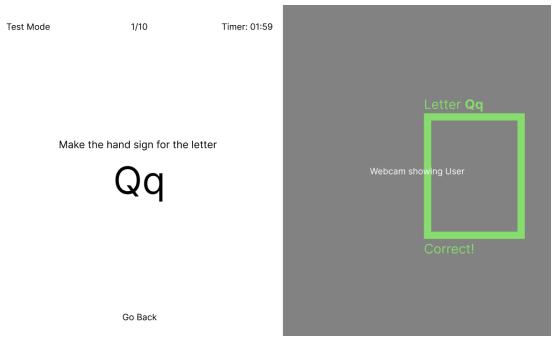

Figure 7: Figma prototype of the Test page

2.1.2 Sign Language Recognition, SLR

The part of the program performing the sign language recognition, was developed through a series of steps shown in Figure 8 including collecting and preprocessing data for the dataset, training and testing the classification model and finally implementing the model with the rest of the program with the GUI.

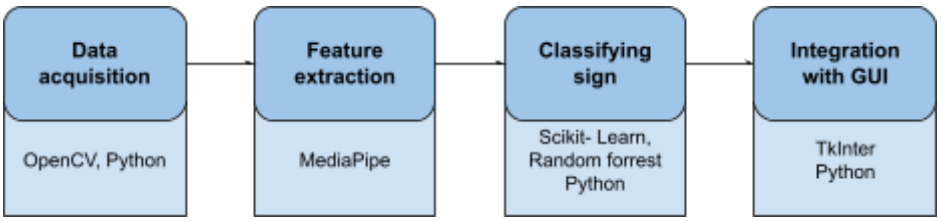

Figure 8: Steps of the gesture recognition process

The process began with creating a dataset containing images of hands signing the 26 ASL letters. In total, 500 pictures signed by two different individuals were taken for every letter, each depicting the hand at a slightly different angle. The images were captured using a MacOS laptop webcam, as seen in Figure 9. A Python script, utilizing the OpenCV library, was used to automate the process of taking pictures. OpenCV is an open-source computer vision library (OpenCV, 2018). As the images taken were of equal size and good quality, no preprocessing was needed to enhance quality or remove noise.

Figure 9: Examples of letters captured with the webcam

To improve the efficiency and accuracy of the model to be trained, the dataset was processed with Google MediaPipe for feature extraction. MediaPipe is a collection of tools and libraries used to integrate ML and AI solutions into different applications (MediaPipe Solutions Guide, n.d.). In particular, MediaPipe Hands was applied. MediaPipe Hands takes an image as input and uses machine learning to detect if a hand is present. If a hand is found, it returns a tensor containing coordinates of 21 points that correspond to the positions of different parts of the hand (Hand Landmarks Detection Guide | MediaPipe, 2023).

To classify the ASL letters, the Random Forest machine learning algorithm was used. This algorithm, in particular, was chosen since it has proven to be effective in other sign language detection programs (Adhikary, et al., 2021). The dataset of extracted features was divided into an 80/20 train-test split. To train and test the Random Forest model, the open-source

library Scikit-learn was used (About Us — Scikit-Learn 0.23.2 Documentation, n.d.). With the test dataset, the accuracy score was computed to 99.6 percent.

The model was integrated into the rest of the program to classify the letters signed by the user. To improve the robustness and accuracy of the classifications in action, which are made four times per second, the program only provides a final classification, which further on will be called the recognized letter, if the latest one is also the most common one for the past two seconds. It therefore utilizes probability to enhance the performance of the classifications. For example, if the program has recognized the letter *A* by having classified it seven times in the past two seconds but the current frame is classified to be another letter, the program waits before changing the recognized letter. This is to see if the following frames will be predicted as that other letter too, then changing the recognized letter to that one when it is the most commonly classified one, or if it continues to classify *A* most commonly. In the latter case, saving the program from an occasional incorrect classification by looking at the majority of the most recent classifications, since the probability of a classified letter being the actual signed letter is lower if it is a deviant one.

2.2 Usability Tests

To evaluate the program, a usability test was designed. The test was conducted with four participants. Each participant was asked to perform three tasks while employing the think-aloud method and to afterwards answer a few questions regarding the experience in the format of a semi-structured interview to gain further insights. The tasks and questions were designed to determine whether or not the goal of the program, to be an effective tool for learning the ASL alphabet, is fulfilled.

One group member acted as an observer and interviewer for each session, filling in the protocol shown in Table 1.

Participant 1			
	Tasks:	**Completed task? (Y/N)**	**Observer's comment:**
Task 1	Practise how to sign the letter A		
Task 2	Practise how to sign the letters C and R		
Task 3	Now without the help of the program: Sign the letters A, C and R		
	Interview questions:	**Answers:**	
Question 1	How did you know that you had signed the letter correctly when using the program?		
Question 2	Do you think that this program can be used to learn sign language?		

Question 3	Did you find anything confusing or unclear when using the program?	
Question 4	Other comments or thoughts:	

Table 1: Protocol template for the usability tests

3. Results

3.1 Final Product

The final program, shown in a demo video here, includes a window divided into two parts. The left part is always showing video from the webcam, while the right part displays different pages with instructions and buttons for user actions. When starting the program, the first page the user is presented with is the Home page (see Figure 10). If pressing the button *Practice Signing*, the user is redirected to the page shown in Figure 11, where the user can select a letter to learn to sign. When selecting a letter, for instance, *A*, the page in Figure 12 is shown. To the right, the user can see an illustration of how to sign the letter as well as a button for returning to the previous page. When the program detects a hand in the image from the webcam, a rectangle pops up around the hand. If the user is signing the letter incorrectly, the rectangle is gray. But if the user is signing it correctly the rectangle changes color to green and a text telling the user that the letter has been learned is shown (see Figure 13). To learn to sign a new letter, the user clicks on the back button. The button with the successfully learned letter has now changed color from blue to green, enabling the user to keep track of which letters they have mastered.

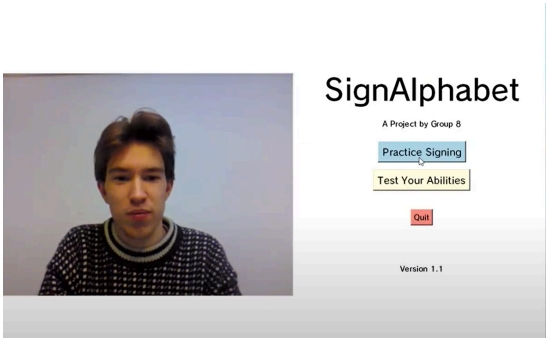

Figure 10: Home Page

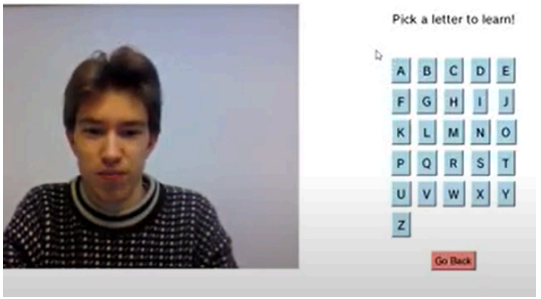

Figure 11: Alphabet Page

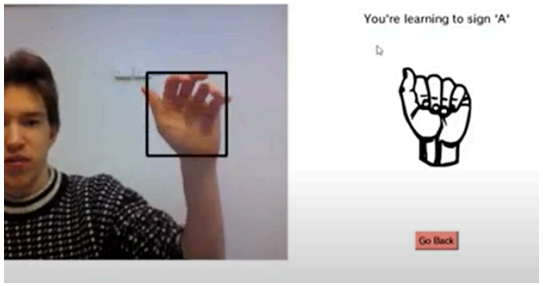

Figure 12: Practice page when signing incorrectly

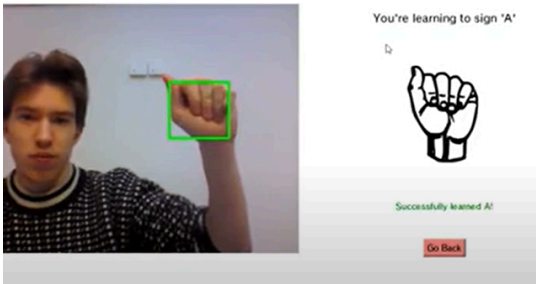

Figure 13: Practice page when signing correctly

The other button on the Home page, *Test Your Abilities*, takes the user to the page shown in Figure 14 (if three or more letters have been learned). Here, the user is tested on three of the letters that have been learned, by prompting the user to do the signs one at a time without any guidance from the program. The next letter sign is only asked for after the previous letter has been signed correctly. The button *Give up* enables the user to go directly back to the Home page. If the user clicks on *Test Your Abilities* and has learned fewer than three letters, the user is taken to the page shown in Figure 15. This is to direct users to the Practice mode. Test mode has been redesigned from the Figma prototype to only test the user on letters successfully learned in Practice mode, visualized by the green letter buttons on the Alphabet page. For seasoned users, clicking on the button *Let's do it anyway!* will start a version of Test page dubbed "*Expert Test*", quizzing the user on five randomly selected letters with a time limit of sixty seconds.

Figure 14: Test page

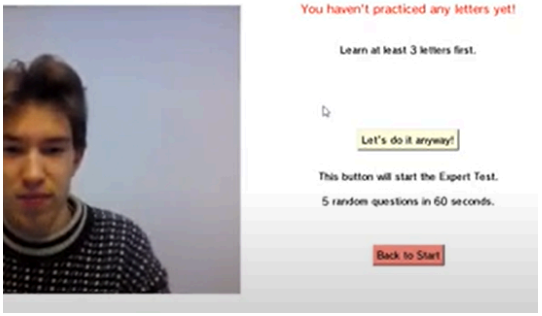

Figure 15: Warning/Expert Test

The final classification model implemented into the program was Random forest, trained on the dataset generated in this study, with feature extraction using MediaPipe's hand landmarker. When tested, it received an accuracy score of 99.6%.

3.2 User Testing

All four participants successfully completed the tasks they were given by the test facilitator. They all managed to practice and test themselves on a couple of ASL letters. The feedback received was overall positive. All users stated that they thought the system could be used to learn the ASL alphabet. They also found the application pleasant and intuitive to interact with.

Two users thought the system needed to give clearer feedback when the correct letter was signed. One user pointed out that it was a bit challenging to understand what angle different signs were supposed to be signed in, demanding clearer instructions on how to sign the signs.

4. Discussion

4.1 The Modalities

The input modalities for this project were gestures and clicks to control the interface. The group first decided that sign language would be interesting to work with, so the use of gestures was natural. The addition of a complementary input modality, namely clicks, was chosen to allow the user to control the interface. It was chosen because it is simple to implement and use, and can be used by a vast majority of people including people with hearing or speaking impairments.

The output modality chosen is visual since sign language is a visual language. The group initially wanted to implement sound to create a redundant relationship between the modalities and send clear feedback to the user about whether they signed correctly or not, reinforcing the visual queue. However, because of time limitations and the fact that this would only benefit hearing users, this was not implemented in the final product.

Although our application could have been developed using only one modality, it would have negatively impacted the usability of the system. Mixing modalities was key to the perceived enjoyment of using the system and for the success of the project as a whole.

4.2 Strengths & Weaknesses of the Program

One strength of the program is the highly performing sign recognition, important for providing valuable feedback regarding sign accuracy. As described in the result, the machine learning model achieved an accuracy score of 99,6%. This is a higher result compared to the study by Adhikary et al. (2021), which this project took inspiration from, with a score of 97,4%. One possible reason for this improved accuracy, even though our model was trained with 27 classes compared to theirs with eleven, is that we were able to look at only the hand compared to both the hands and body since we only detect letters and not words or phrases like in their study. Another difference between our study and theirs is that their program uses the Indian Sign Language, not the American. Additionally, they did not develop a program intended for users with a graphical user interface as we did, instead, they focused on testing and comparing different techniques and machine learning models.

After advice from our supervisor, we chose to create our own dataset with pictures captured from videos when people were making the signs. This resulted in a dataset well adapted to the context of the program, which is a strength since it provided the foundation for the robustness of the gesture recognition.

The user testing indicated that our application served its purpose of providing a fun and easy way for people to start learning fingerspelling. It also highlighted that one strength of the system is that it is intuitive and simple to use. A weakness pointed out by two users was that the feedback provided when signing the correct sign was insufficient. Therefore, after the user tests, we implemented clearer visual feedback by complementing the previous green box shown around the hand with a green text saying: “*Successfully learned A*”.

There were also weaknesses in the user testing itself. The testing was done on a very limited number of participants. Recruiting more participants might have led to discovering more problems with our interface. Another weakness is that we didn't get any feedback or input from someone who knows sign language. Making more of an "expert evaluation" might have provided valuable feedback, as well as involving a sign language expert early on in the development process to give suggestions on the design of the application.

4.3 Further Development

The program as postulated in the initial stages of development was successfully realized in the final prototype. We managed to create the two modes (practice and test), a recognition system, and an interface in the exact style of the storyboards and Figma sketches.

Our approach when developing *Sign Alphabet* was bare-bones and minimalistic, where the main focus was on core functionality and fulfilling the requirements of the system. This was decided when considering the limited window of development time. During development, and even when writing this report, we discovered more and more potential in the concept of *Sign Alphabet*, but still decided to stick to realizing only the original plan for the system. This choice gave us time to go over the many possible changes and future improvements, had the time window been lengthened or if we had changed plans along the way.

Starting with changes to the program as it exists in the current state: letters J and Z require hand gestures that are similar to other signs, the only difference being that they are signed by moving the gesture in a pattern. As our classifying method used individual static images to classify a sign, signs for I and J, which use the same finger positions, are practically the same. Therefore, finding a workaround for the classification of J and Z would improve the system.

Limiting the number of signs the user can practice is a minor tweak that can help users remember each sign better. This method is used in language learning applications such as *Duolingo* (Duolingo, 2019), where users learn a few words per module, and must remember them well before being able to move on.

In regards to usability, there has been no onboarding process installed to introduce users to the functionality of the system. An easy fix would be to ask the user to hold up their hand in a certain position to show the user it has been detected.

Fundamental changes include changing the test mode to provide a more dynamic implementation of the gesture modality. The system, in its current iteration, only serves to showcase the modality in a single sense, and the practice and test modes work in more or less the same way. Alternative ways of testing the user could not only strengthen the dynamism of the use of gestures but also the satisfaction of the user in applying their knowledge and puzzle-solving skills. Possible ways include: asking the user to finger-spell words or names, doing various tasks like answering a quiz, controlling a character in a game, or expressing themselves while using fingerspelling. The instructions given to the user when learning each sign could also benefit from further using the data that the MediaPipe recognition system provides: landmarks could communicate to the user which fingers are in the correct position and how to go from letter to letter when fingerspelling.

We also realized that when arguing for the "democratization of sign language learning", the best way to showcase it would not be through a Tkinter program; but rather a web page that is readily available on the internet. Therefore, *Sign Alphabet* achieves its goal of being the "easy and accessible way to learn sign language" only when it is put in an open, global arena. Seen through this lens, it became apparent to us that such an implementation only expands the potential of such a service, allowing global collaborators to create versions of the system intended for different sign languages.

5. Conclusion

In this project we set out to create a multimodal gesture recognition system for ASL alphabet learning. We developed the system Sign Alphabet which uses gestures and touchpad clicks as input modalities and gives visual output to the user. The system aimed at lowering thresholds for starting to learn sign language, as well as exploring the modality and potential use cases for gesture recognition.

The finished system proved that using gestures in combination with clicks and visual output for learning the ASL alphabet works. Considering the small-scale execution of *Sign Alphabet*, combining gestures with other modalities and systems in a more dynamic way can further solidify the potential of the modality to improve sign language pedagogy in the future.

Bibliography

About us — scikit-learn 0.23.2 documentation. (n.d.). Scikit-Learn.org. https://scikit-learn.org/stable/about.html#history

Adeyanju, I. A., Bello, O. O., & Adegboye, M. A. (2021). Machine learning methods for sign language recognition: A critical review and analysis. *Intelligent Systems with Applications*, *12*, 200056. https://doi.org/10.1016/j.iswa.2021.200056

Adhikary, S., Talukdar, A. K., & Kumar Sarma, K. (2021, November 1). *A Vision-based System for Recognition of Words used in Indian Sign Language Using MediaPipe*. IEEE Xplore. https://doi.org/10.1109/ICIIP53038.2021.9702551

Duolingo. (2019). *Learn a Language for Free*. Duolingo. https://www.duolingo.com/

Foggetti, F. (2022, February 17). *5 Interesting Facts about Sign Languages - Hand Talk*. https://www.handtalk.me/en/blog/nteresting-facts-about-sign-languages/)

Joy, J., Balakrishnan, K., & Sreeraj, M. (2019). SignQuiz: A Quiz Based Tool for Learning Fingerspelled Signs in Indian Sign Language Using ASLR. *IEEE Access*, *7*, 28363–28371. https://doi.org/10.1109/access.2019.2901863

MediaPipe Solutions guide. (n.d.). Google Developers. https://developers.google.com/mediapipe/solutions/guide

OpenCV. (2018). *About OpenCV*. OpenCV. https://opencv.org/about/

Python Software Foundation. (2019). *tkinter — Python interface to Tcl/Tk — Python 3.7.2 documentation*. Python.org. https://docs.python.org/3/library/tkinter.html

Reagan, T. (2007). SIGN LANGUAGE AND LINGUISTIC UNIVERSALS, Wendy Sandler and Diane Lillo-Martin. *Studies in Second Language Acquisition*, *29*(04). https://doi.org/10.1017/s0272263107070507

Rosen, R. S. (2019). *The Routledge Handbook of Sign Language Pedagogy*. Routledge.

S. M. Kamal, Y. Chen, S. Li, X. Shi and J. Zheng, "Technical Approaches to Chinese Sign Language Processing: A Review," in IEEE Access, vol. 7, pp. 96926-96935, 2019, doi: 10.1109/ACCESS.2019.2929174.

Hand landmarks detection guide | MediaPipe. (2023). Google Developers. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker